Computer Architecture Lab

(EL-2012)

LABORATORY MANUAL

FALL 2026



(Lab#08)

Implementation of Program Counter and Instruction Memory

Student Name: _____ Roll No: _____ Section: _A__

Date performed: _____, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Learning Objectives:

- To understand the purpose of the Program Counter (PC) and Instruction Memory (IM).
- Simulate how a PC fetches instructions step-by-step.
- To implement PC and IM and simulate using ModelSim.

Tool Used:

ModelSim

Introduction:

1. **Program Counter (PC):**

   A program counter is a register in a computer processor that contains the address (location) of the instruction being executed at the current time. As each instruction gets fetched, the program counter increases its stored value by 1. After each instruction is fetched, the program counter points to the next instruction in the sequence. When the computer restarts or is reset, the program counter normally reverts to 0. There are two different ways of implementing a Program Counter which will be design further in the manual.

2. **Instruction Memory (IM):**

   The next part of a processor after the program counter is the instruction memory. The program counter sends its output to the instruction memory, which is the address of the instruction stored there. Instruction memory typically has one input (address) and one output (instruction). The number of address bits depends on the processor. For example, if the processor is 8-bit, the instruction line is usually 8 bits wide. The total size of instruction memory is 2^n * m, where "n" is the number of address bits and "m" is the width of the instruction. The design of instruction memory will be implemented in the manual.

The program counter and instruction register, both different types of registers, play a vital role in a processor's instruction cycle. A register is a small data holding place that the processor can access quickly.

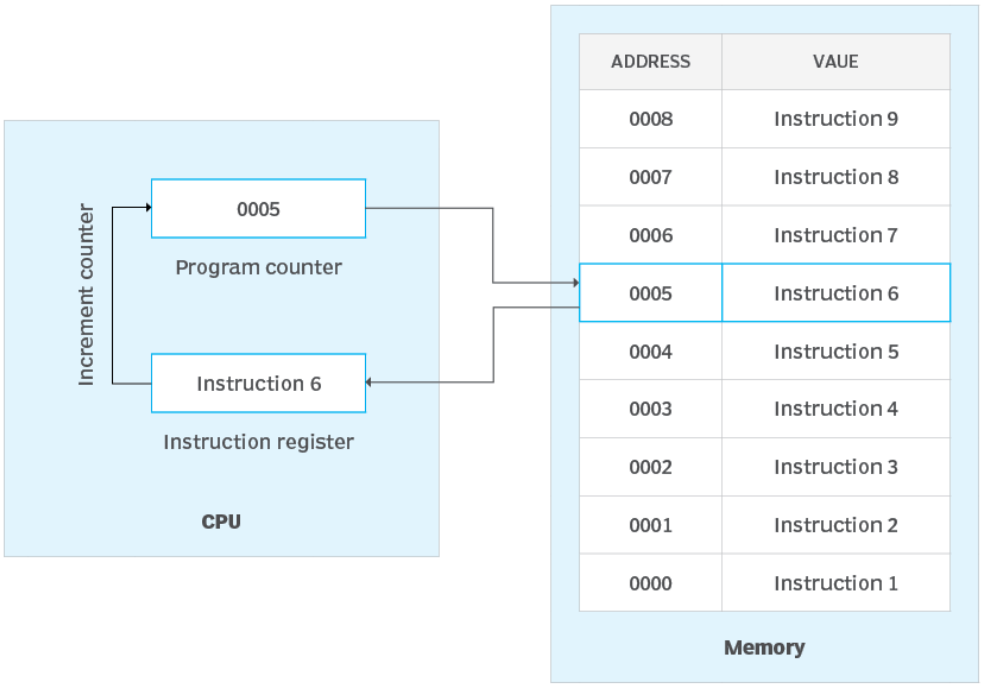
The CPU knows which instruction to fetch from memory by referencing its address in the program counter. The instruction register holds the instruction after it is fetched from memory. Once the instruction is in the instruction register, the CPU can decode and execute it. In this sense, the instruction register is dependent on the program counter to ensure that the instruction register contains the correct instruction at any given time.

During a normal instruction cycle, the CPU fetches, decodes and executes one of the program's instructions. Before fetching the instruction from memory, the CPU retrieve's the instruction's address from the program counter. In this way, the CPU knows which instruction to retrieve. For example, if the program counter shows an address of 0851, the CPU knows that it should

retrieve the program instruction located at the 0851 memory address.

After determining the correct address, the CPU fetches the instruction, places it into the instruction register, and increments the program counter by 1. The CPU then decodes the instruction in the instruction register. If the instruction is complete, the CPU executes the instruction and continues with the next instruction cycle.

The following figure provides a simple overview of the roles that the program counter and instruction register play in the instruction cycle. Here, the program counter shows a memory address of 0005. This causes the CPU to fetch Instruction 6 from memory and load it into the instruction register. The CPU then updates the program counter, decodes Instruction 6, and executes the instruction. The program counter should now read 0006, which points to the next instruction.



The above figure shows the CPU and memory at their most basic. It does not include components such as the CPU's accumulator or arithmetic logic unit (ALU). The figure also indicates that each instruction fits neatly into a single memory space. In many cases, however, instructions are too large for a single memory location and must be split across multiple addresses.

Modern processors easily accommodate instructions that span multiple memory addresses. When the CPU decodes an instruction in the instruction register, it determines whether the instruction is complete. If it's not, it will fetch the next part of the instruction, again referring to the program counter. After all the instruction's parts are fetched into the instruction register, the CPU will execute the instruction in its entirety and then continue to the next instruction cycle.

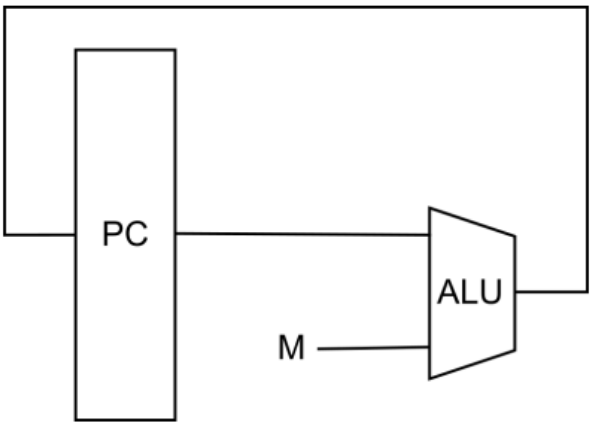
Another situation that a CPU routinely encounters is a program's control flow logic. Such logic typically requires the processor to pull its instructions from memory in a non-sequential order. For example, the program might include a function call or conditional loop that requires access to specific instructions that are not next in line, because they're stored in memory.

When the CPU encounters this issue, it updates the program counter to the location specified by the current instruction. Then, when the CPU is ready to launch the next instruction cycle, it can fetch the address from the program counter because it now points to the correct location in memory.

If a computer is restarted or reset, the program counter normally reverts to 0.

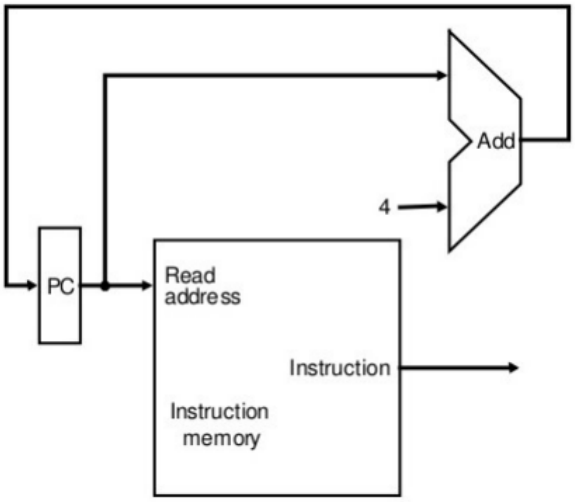
Design:

The main idea of program counter is to increment the (output) address after every clock cycle. Now there are two possibilities by which the PC can increment its output based on the type of architecture used. Harvard-based systems tend to store one machine word per memory location. This means that every cycle the PC needs to be incremented by 1. Computers that share data and instruction memory together typically are byte addressable, which is to say that each byte has its own address, as opposed to each machine word having its own address. In these situations, the PC needs to be incremented by the number of bytes in the machine word.



In the design shown above, the PC block is a simple register with a reset and clock signal. ALU is an adder, adding two inputs. One is the output from the register and second one is the amount M by which we want to increment our PC as discussed above. Final output will be provided by the ALU/ADDER which is also a feedback to the register.

Design of an Instruction Memory is simple, there is one input fed from the PC and one output (instruction) which is further decoded and executed.



Setting up Program Counter:

- Open ModelSim.
- Click on File → New → Project.
- Name your project (e.g., "PC_IM_Simulation").
- Choose the appropriate directory to store the project files.
- Create Verilog file for Program Counter and name your file (e.g., "program_counter.v").

Write the following Verilog code to implement the Program Counter:

```
module program_counter (
    input clk,          // Clock input
    input reset,        // Reset signal
    input [7:0] addr,   // Address input
    output reg [7:0] pc_out // Program counter output
);

    always @(posedge clk or posedge reset) begin
        if (reset)
            pc_out <= 8'b00000000;  // Reset to address 0
        else
            pc_out <= addr;         // Update PC with the new address
    end
endmodule
```

The Program Counter (PC) stores an 8-bit address (which is the location of the next instruction). The clk signal triggers the PC to update the address when it changes. The reset signal resets the PC to 0. The input addr represents the next address to be loaded into the PC.

Setting up Instruction Memory:

- Create Verilog file for Instruction Memory
- Click on File → New → Source.

- Select Verilog and click on OK and name your file (e.g., "instruction_memory.v").

OR

- You can use the same file for instruction memory (specifically in ModelSim).

Write the following Verilog code for the Instruction Memory:

```
module instruction_memory (
    input [7:0] addr,        // Address input
    output reg [7:0] instruction  // Instruction output
);

    // Memory initialization with 4 instructions
    reg [7:0] memory [0:15];  // 16x8-bit memory

    initial begin
        memory[0] = 8'b00000001; // Instruction 1
        memory[1] = 8'b00000010; // Instruction 2
        memory[2] = 8'b00000011; // Instruction 3
        memory[3] = 8'b00000100; // Instruction 4
    end

    always @(addr) begin
        instruction = memory[addr]; // Fetch instruction from memory
    end
endmodule
```

The instruction memory is a simple array that holds instructions. The memory size is 16 addresses, and each address holds an 8-bit instruction. The addr input is used to fetch the instruction from memory.

The testbench instantiates both the Program Counter and Instruction Memory. The clock signal is generated continuously, and the reset signal is applied initially. The address for the Program Counter is changed every few time units, simulating the fetching of different instructions.

```
module tb_program_counter;

    reg clk;                // Clock signal
    reg reset;              // Reset signal
    reg [7:0] addr;         // Address for Program Counter
    wire [7:0] pc_out;      // Program Counter output
    wire [7:0] instruction; // Instruction output from memory
```

```
    // Instantiate Program Counter
    program_counter pc (
        .clk(clk),
        .reset(reset),
        .addr(addr),
        .pc_out(pc_out)
    );

    // Instantiate Instruction Memory
    instruction_memory im (
        .addr(pc_out),          // Use PC output as address
        .instruction(instruction)
    );
    // Generate Clock signal
    always begin
        #5 clk = ~clk;          // Clock with period of 10 time units
    end
    // Initialize and apply inputs
    initial begin
        // Initialize signals
        clk = 0;
        reset = 0;
        addr = 0;

        // Reset the program counter
        reset = 1;
        #10 reset = 0;

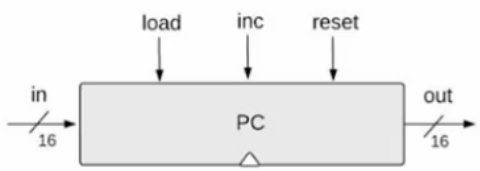
        // Simulate Program Counter fetching instructions
        #10 addr = 1;
        #10 addr = 2;
        #10 addr = 3;
        #10 addr = 0;
    end
endmodule
```

Tasks:

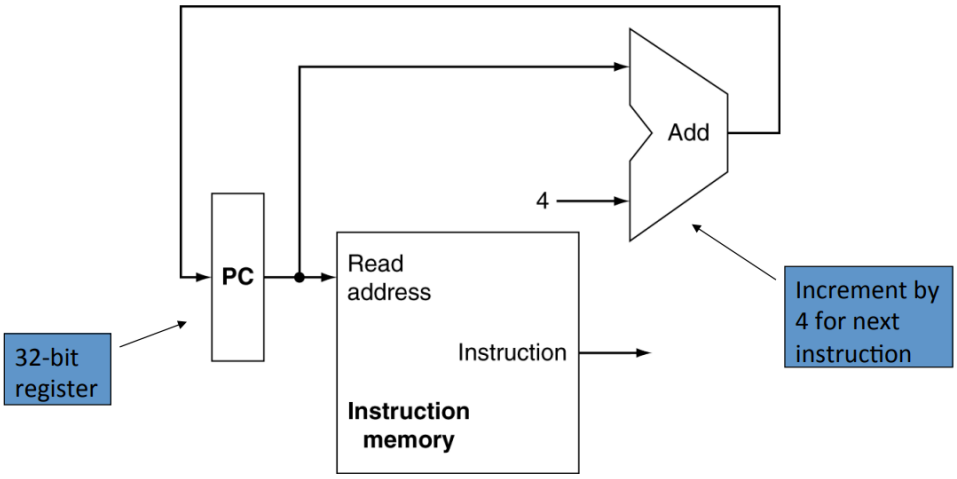
Complete the following tasks in this Lab.

1. Implement a program counter to support a 4-bit address. Simulate and verify the results using the same testbench.
2. Simulate the Program Counter increments automatically by 1 after every clock cycle (instead of setting the address manually). Verify that the instructions are fetched sequentially from the instruction memory.

Add error handling in the Instruction Memory module to return a default instruction (e.g., 8'b00000000) if an invalid address is provided.



3. Implement a 32-bit PC register compliant with the RV32 architecture. To make simulation manageable within the test bench environment, configure the PC's sequential increment logic to add 4 bytes (+4) on every rising clock edge, ensuring proper 32-bit word alignment. Truncate or use a bit-splitter on the lower 4 bits of the PC output to drive a 4-bit address space for your immediate test environment. Simulate the design using the provided testbench. Verify that the PC resets correctly to 0x00000000 (or your defined base address) and holds its value stably between clock edges.
4. Integrate the 32-bit PC with a 32-bit wide Instruction Memory (IM) module. The circuit must automatically increment the PC by 4 after every clock cycle (simulating sequential execution without manual address intervention). Load a sample program into the Instruction Memory. Run the clock continuously and verify via the test bench waveforms that 32-bit instructions are fetched sequentially from memory addresses (0x00, 0x04, 0x08, 0x0C, etc.) matching the expected program flow.



5. Simulate a 32-bit RISC-V PC and an IM module using Logisim.